REVISION CHEAT SHEET:

PIPELINE / ROUTING

ASP.NET Core Request Pipeline, Middleware, Routing and Error Handling

Interview-ready notes · examples · C# · diagrams · clickable index

INDEX / TABLE OF CONTENTS

Click a section name to jump to that section.

1. Where This Topic Fits
2. What Is Middleware?
3. Basic Middleware Mental Model
Diagram 1 — Request and Response Flow
4. Simple Inline Middleware
5. RequestDelegate
6. Why Middleware Is Asynchronous
7. HttpContext
8. HttpRequest
9. HttpResponse
10. HttpContext.User
11. HttpContext.RequestServices
12. HttpContext.Items
13. HttpContext Is Request-Scoped Data
PART 1 — USE, RUN, AND MAP
14. app.Use
15. app.Run as Terminal Middleware
16. Example of Terminal Behavior
17. Short-Circuiting
18. Why Short-Circuit?
19. Do Not Call next After Sending Response
20. Response.HasStarted
21. app.Map
22. MapWhen
23. UseWhen
24. Use vs Run vs Map
PART 2 — MIDDLEWARE ORDERING
25. Middleware Order Is Critical
26. Typical API Pipeline
27. Why Exception Handling Is Early
28. Before and After Middleware
29. Example Logging Middleware
30. Register Custom Middleware
PART 3 — CUSTOM MIDDLEWARE LIFETIMES
31. Conventional Middleware Is Constructed Once
32. Do Not Inject Scoped Service Into Conventional Middleware Constructor
33. Inject Scoped Dependency Into InvokeAsync
34. IMiddleware
35. Conventional Middleware vs IMiddleware
PART 4 — ROUTING FUNDAMENTALS
36. What Is Routing?
37. What Is an Endpoint?
38. Endpoint Routing Mental Model
39. Basic Minimal API Route
40. Controller Attribute Routing
41. Route Template Components
42. Literal Route Segments
43. Route Parameters
44. Optional Parameters
45. Default Route Values
46. Catch-All Parameters
47. Route Constraints
48. Common Constraint Examples
49. Route Constraints Are Not Input Validation
50. Route Specificity
51. Route Precedence Rules
52. Ambiguous Routes
53. HTTP Method Matching
54. Attribute Routing in Web APIs
55. Route Tokens
56. Prefer Stable Resource-Oriented API Routes
57. Route Groups

58. Route Metadata
59. UseRouting in Modern WebApplication
60. Endpoint-Aware Middleware
61. ShortCircuit Endpoint Support
PART 5 — ERROR HANDLING
62. Why Centralized Error Handling Matters
63. Expected Error vs Unexpected Exception
64. Developer Exception Page
65. Never Expose Stack Traces in Production
66. UseExceptionHandler
67. Problem Details
68. Basic Problem Details Configuration
69. IExceptionHandler
70. Register IExceptionHandler
71. GiftNotFoundExceptionHandler
72. Exception Handler Ordering
73. Important .NET 10 Exception Diagnostics Change
74. Why This Matters
75. Exception-to-HTTP Mapping
76. 400 vs 404
77. 401 vs 403
78. 409 Conflict
79. 500 Internal Server Error
80. ProblemDetails Customization
81. Status Code Pages
82. Exception vs Status Code
83. Don't Throw Exceptions for Every Expected Validation Failure
84. Exception Middleware Cannot Rewrite Started Response
85. Error Handler Must Be Reliable
PART 6 — COMPLETE FUNDRAISING PIPELINE
86. Program.cs Example
87. Request Example
Diagram 2 — Full Request Pipeline
88. Successful Request
89. Gift Not Found
90. Unexpected Failure
PART 7 — ROUTING DESIGN FOR THE FUNDRAISING API
91. Gift Endpoints
92. Campaign Endpoints
93. Fund Endpoints
94. Nested Resources
95. Route IDs and Tenant Security
PART 8 — PERFORMANCE
96. Every Middleware Adds Work
97. Put Cheap Rejections Early When Safe
98. Avoid Expensive Work Before Routing Unless Every Request Needs It
99. Avoid Reading Large Request Body in Generic Middleware
100. Routing Performance
PART 9 — SECURITY
101. Middleware Order Can Be a Security Boundary
102. CORS Ordering Matters
103. Don't Trust Route Values
104. Don't Log Sensitive Request Data
105. Production Error Responses Must Be Safe
PART 10 — TESTING
106. Unit Test Middleware?
107. Integration Tests Are Often Better for Pipeline Behavior
108. Routing Tests
109. Error Handler Tests
PART 11 — COMMON MISTAKES
110. Wrong Middleware Order
111. Exception Handler Too Late
112. Calling next After Terminal Response
113. Using Route Constraint as Validation
114. Catching Exception in Every Controller
115. Returning Exception.Message to Clients
116. Using Exception for Every 404
117. Injecting Scoped DbContext Into Conventional Middleware Constructor
118. Giant Custom Middleware
119. Business Logic in Middleware
120. Storing Per-Request Data in Singleton Middleware Fields
121. Writing Response Then Throwing
122. Overusing Branching Middleware
PART 12 — BEST PRACTICES
PART 13 — PRACTICAL EXERCISES
Exercise 1 — Before and After Middleware
Exercise 2 — Short-Circuit
Exercise 3 — Request Timing
Exercise 4 — Scoped Middleware Dependency
Exercise 5 — IMiddleware
Exercise 6 — Route Precedence

PIPELINE / ROUTING

Exercise 7 — Route Constraint
Exercise 8 — Centralized 404 Exception
Exercise 9 — Unexpected Exception
Exercise 10 — Status Code Pages
Exercise 11 — Pipeline Ordering
Exercise 12 — API Integration Test
PART 14 — EXACTLY 25 INTERVIEW QUESTIONS
1. What is middleware in ASP.NET Core?
2. How does the ASP.NET Core request pipeline work?
3. What is HttpContext?
4. What is the difference between app.Use and app.Run?
5. What is middleware short-circuiting?
6. Why is middleware order important?
7. Why should authentication normally come before authorization?
8. How do you create custom middleware?
9. Can a conventional middleware constructor inject a scoped service?
10. What is IMiddleware?
11. What is routing in ASP.NET Core?
12. What is an endpoint?
13. What is a route template?
14. What is a route constraint for?
15. How does ASP.NET Core choose between competing routes?
16. What is the difference between MapControllers and MapControllerRoute?
17. Do modern ASP.NET Core applications always need explicit UseRouting and UseEndpoints?
18. What is the Developer Exception Page?
19. What does UseExceptionHandler do?
20. What is IExceptionHandler?
21. What changed for handled-exception diagnostics in .NET 10?
22. What is ProblemDetails?
23. What is the difference between exception handling and status code pages?
24. Why can't exception middleware always replace an error response after an exception?
25. How would you design the Fundraising Platform request pipeline?
QUICK REVISION
FINAL END-TO-END REQUEST FLOW

PIPELINE / ROUTING

1. Where This Topic Fits

In Topic 37, the application reached this point:

EXAMPLE / FLOW

```
Process starts
    ↓
Program.cs
    ↓
WebApplicationBuilder
    ↓
Register services
    ↓
builder.Build()
    ↓
Configure application
    ↓
app.Run()
    ↓
Kestrel starts listening
```

Now the application is running.

The next question is:

> **What happens when an HTTP request actually reaches the application?**

Suppose a client sends:

EXAMPLE / FLOW

```
POST /api/gifts/125/complete
```

PIPELINE / ROUTING

The request doesn't jump directly from Kestrel to `GiftsController`.

It moves through an ordered **request-processing pipeline**.

Conceptually:

EXAMPLE / FLOW

```
Client
   ↓
Kestrel
   ↓
Exception Handling
   ↓
HTTPS / Security
   ↓
Routing
   ↓
Authentication
   ↓
Authorization
   ↓
Custom Middleware
   ↓
Selected Endpoint
   ↓
Controller / Minimal API
   ↓
Application Service
   ↓
Response travels back outward
```

That pipeline is one of the central concepts in ASP.NET Core.

2. What Is Middleware?

Middleware is software assembled into an ASP.NET Core pipeline to process HTTP requests and responses.

Each middleware component may:

EXAMPLE / FLOW

```
Inspect the request

Modify the request

Perform work before the next component

Call the next component

Perform work after the next component

Modify the response

Stop the pipeline completely
```

Microsoft describes middleware as components assembled into an application pipeline where each component can perform work and optionally invoke the next component. The order in `Program.cs` determines request order, while response processing unwinds in reverse order.

3. Basic Middleware Mental Model

Suppose the pipeline contains:

EXAMPLE / FLOW

```
Middleware A
Middleware B
Middleware C
Endpoint
```

Request flow:

EXAMPLE / FLOW

```
Request
   ↓
A
   ↓
B
   ↓
C
   ↓
Endpoint
```

Response flow:

EXAMPLE / FLOW

```
Endpoint
   ↓
C
   ↓
B
   ↓
A
   ↓
Response
```

Therefore middleware behaves much like nested calls:

EXAMPLE / FLOW

```
A(
    B(
        C(
            Endpoint()
        )
    )
)
```

This explains why code **after** `await next(...)` executes while the response travels back through the pipeline.

Diagram 1 — Request and Response Flow

EXAMPLE / FLOW

```
CLIENT
  │
  ▼
Middleware A
  │ before
  ▼
Middleware B
```

```
  | before
  ▼
Middleware C
  | before
  ▼
ENDPOINT
  |
  ▼
Middleware C
  | after
  ▼
Middleware B
  | after
  ▼
Middleware A
  | after
  ▼
CLIENT
```

This request-forward/response-reverse behavior is fundamental.

4. Simple Inline Middleware

Example:

C#

```
app.Use(
    async (
        context,
        next) =>
    {
        Console.WriteLine(
            "Before next middleware.");

        await next(
            context);

        Console.WriteLine(
            "After next middleware.");
    });
```

For one request:

EXAMPLE / FLOW

```
Before next middleware
    ↓
rest of pipeline executes
    ↓
After next middleware
```

The `next` parameter represents the next `RequestDelegate` in the pipeline.

5. RequestDelegate

Conceptually, ASP.NET Core middleware is built around:

C#

```
RequestDelegate
```

which is effectively an asynchronous function that receives:

```
C#
HttpContext
```

and returns:

```
C#
Task
```

Conceptually:

```
C#
Task HandleRequestAsync(
    HttpContext context);
```

A middleware receives the next delegate and decides whether to invoke it.

6. Why Middleware Is Asynchronous

HTTP processing frequently performs I/O:

```
EXAMPLE / FLOW
Authentication database lookup

Distributed cache

Remote API call

Reading request body

Writing response

Database query
```

Therefore middleware is normally asynchronous:

```
C#
public async Task InvokeAsync(
    HttpContext context)
{
    await _next(
        context);
}
```

The same `async` / `await` principles from Topic 27 apply here.

Do not block request threads using:

```
C#
.Result
```

or:

```
C#
.Wait()
```

inside middleware.

7. HttpContext

Every HTTP request has an:

```C#
HttpContext
```

It represents the current request/response environment.

Important members include:

```C#
context.Request

context.Response

context.User

context.RequestServices

context.Items

context.Features

context.Connection

context.TraceIdentifier
```

`HttpContext` gives middleware and endpoints access to request and response information, while the `Features` collection exposes lower-level server/request capabilities.

8. HttpRequest

Access using:

```C#
HttpRequest request = context.Request;
```

Common properties include:

```C#
request.Method

request.Path

request.Query

request.Headers

request.Cookies

request.Body

request.ContentType

request.Scheme

request.Host

request.RouteValues
```

Example:

```
C#
string method = context.Request.Method;

PathString path = context.Request.Path;
```

9. HttpResponse

Access:

```
C#
HttpResponse response = context.Response;
```

Important members:

```
C#
response.StatusCode

response.Headers

response.ContentType

response.Body

response.HasStarted
```

Response headers and status normally must be set **before** response output begins. Once response headers/body have started, ASP.NET Core can no longer safely rewrite status and headers.

10. HttpContext.User

After authentication:

```
C#
context.User
```

contains the request's:

```
C#
ClaimsPrincipal
```

Example:

```
C#
bool authenticated =
    context.User
        .Identity?
        .IsAuthenticated
    == true;
```

Authorization decisions may later use these claims.

We'll cover authentication and authorization deeply in Topic 40.

11. HttpContext.RequestServices

Each HTTP request normally has its own DI scope.

The request-specific service provider is available through:

```C#
context.RequestServices
```

Example:

```C#
IGiftRepository repository =
    context.RequestServices
        .GetRequiredService<
            IGiftRepository>();
```

However, this should not become normal Service Locator usage.

Prefer injecting dependencies into middleware appropriately rather than resolving arbitrary services manually from `RequestServices`.

12. HttpContext.Items

`Items` provides per-request storage.

Example middleware:

```C#
context.Items[
    "CorrelationId"] = Guid.NewGuid();
```

Later:

```C#
Guid correlationId =
    (Guid)context.Items[
        "CorrelationId"]!;
```

The value exists only for the current request.

This can be useful for middleware-to-middleware communication.

Prefer typed abstractions when the data represents an important application concept rather than scattering string keys everywhere.

13. HttpContext Is Request-Scoped Data

Never store the current `HttpContext` in a singleton field:

```C#
public sealed class BadGlobalState
{
    public HttpContext? Current
    {
        get;
        set;
    }
}
```

Why?

Concurrent requests would overwrite each other's state.

`HttpContext` belongs to one request and should not be treated as global application state.

PART 1 — USE, RUN, AND MAP

14. app.Use

`Use` adds middleware that can call the next middleware.

Example:

```
C#
app.Use(
    async (
        context,
        next) =>
    {
        await context.Response
            .WriteAsync(
                "Before\n");

        await next(
            context);

        await context.Response
            .WriteAsync(
                "After\n");
    });
```

Conceptually:

```
EXAMPLE / FLOW
Use
    ↓
can continue pipeline
```

15. app.Run as Terminal Middleware

`Run` registers a terminal delegate.

Example:

```
C#
app.Run(
    async context =>
    {
        await context.Response
            .WriteAsync(
                "Pipeline ended.");
    });
```

It does not receive a `next` delegate.

Therefore:

```
EXAMPLE / FLOW
Run
    ↓
terminal
```

The first `Run` delegate terminates the pipeline, so middleware registered after it isn't reached through that pipeline.

16. Example of Terminal Behavior

C#

```
app.Run(
    async context =>
    {
        await context.Response
            .WriteAsync(
                "First terminal middleware");
    });

app.Run(
    async context =>
    {
        await context.Response
            .WriteAsync(
                "Never reached");
    });
```

Only the first terminal delegate executes.

17. Short-Circuiting

A middleware can terminate processing even when written with `Use`.

Example:

C#

```
app.Use(
    async (
        context,
        next) =>
    {
        if (
            context.Request.Path
            ==
            "/maintenance")
        {
            context.Response.StatusCode =
                StatusCodes
                    .Status503ServiceUnavailable;

            await context.Response
                .WriteAsync(
                    "System maintenance.");

            return;
        }

        await next(
            context);
    });
```

For:

EXAMPLE / FLOW

```
/maintenance
```

the pipeline becomes:

EXAMPLE / FLOW

```
Request
    ↓
Maintenance middleware
    ↓
503 response
    ↓
STOP
```

The endpoint is never called.

18. Why Short-Circuit?

Useful for:

EXAMPLE / FLOW

```
Static files

Maintenance mode

Rejected requests

Rate limits

Health probes

Certain security checks

Known simple responses
```

Short-circuiting can improve performance because unnecessary downstream work is avoided.

19. Do Not Call next After Sending Response

Bad:

C#

```
await context.Response
    .WriteAsync(
        "Access denied");

await next(
    context);
```

The response may already be started.

Downstream middleware may attempt to:

EXAMPLE / FLOW

```
change status

change headers

write incompatible body
```

This can cause exceptions or corrupt/protocol-invalid output.

Microsoft specifically warns against invoking downstream middleware after a response has already been sent.

20. Response.HasStarted

PIPELINE / ROUTING

You can inspect:

```C#
if (
    context.Response
        .HasStarted)
{
}
```

This tells you whether response headers have begun being sent.

This matters particularly in:

```EXAMPLE / FLOW
exception handling
response rewriting
custom error middleware
```

Once the response has started, an error handler generally cannot replace the response cleanly.

21. app.Map

`Map` branches the middleware pipeline by request path.

Example:

```C#
app.Map(
    "/admin",
    adminApp =>
    {
        adminApp.Run(
            async context =>
            {
                await context.Response
                    .WriteAsync(
                        "Admin branch");
            });
    });
```

Request:

```EXAMPLE / FLOW
/admin/users
```

enters:

```EXAMPLE / FLOW
/admin branch
```

while:

```EXAMPLE / FLOW
/api/gifts
```

continues through the normal branch.

Microsoft documents `Map` as a path-based pipeline branching mechanism.

22. MapWhen

Another branching technique is:

```C#
app.MapWhen(
    context =>
        context.Request
            .Query
            .ContainsKey(
                "debug"),
    branch =>
    {
        branch.Run(
            async context =>
            {
                await context.Response
                    .WriteAsync(
                        "Debug branch");
            });
    });
```

This uses a predicate rather than a simple path.

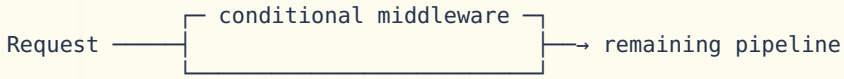
Use only when branching really belongs at middleware level.

23. UseWhen

`UseWhen` creates a conditional branch that can later rejoin the original pipeline.

Conceptually:

EXAMPLE / FLOW



This differs conceptually from terminal path branches.

24. Use vs Run vs Map

API	Purpose
`Use`	Add middleware that may call next
`Run`	Add terminal request delegate
`Map`	Branch pipeline based on path
`MapWhen`	Branch based on predicate
`UseWhen`	Conditionally execute branch and rejoin

PART 2 — MIDDLEWARE ORDERING

25. Middleware Order Is Critical

Consider:

```
C#
app.UseAuthentication();

app.UseAuthorization();
```

Authentication must normally run first because Authorization needs an authenticated:

```
C#
ClaimsPrincipal
```

Similarly, error-handling middleware should usually appear early so it can catch exceptions thrown by downstream middleware.

Microsoft emphasizes that middleware order is critical for security, performance, and functionality.

26. Typical API Pipeline

A simplified API pipeline might be:

```
C#
app.UseExceptionHandler();

app.UseHttpsRedirection();

app.UseRouting();

app.UseCors();

app.UseAuthentication();

app.UseAuthorization();

app.MapControllers();
```

The exact pipeline depends on application needs.

Important ordering relationships may include:

```
EXAMPLE / FLOW
Exception handling
    ↓
HTTPS
    ↓
Routing
    ↓
CORS
    ↓
Authentication
    ↓
Authorization
    ↓
Endpoint
```

Microsoft specifically notes ordering requirements such as CORS before authentication/authorization in relevant pipeline scenarios.

27. Why Exception Handling Is Early

Suppose:

EXAMPLE / FLOW

```
Exception Handler
    ↓
Authentication
    ↓
Authorization
    ↓
Controller
```

If the controller throws:

EXAMPLE / FLOW

```
Controller
    ↑ exception
Authorization
    ↑
Authentication
    ↑
Exception Handler catches
```

If Exception Handler were registered after the controller endpoint execution point, it could not wrap the failing component.

Therefore:

> ⓘ Error-handling middleware should generally be placed early enough to surround downstream processing.

28. Before and After Middleware

Middleware:

C#

```
app.Use(
    async (
        context,
        next) =>
    {
        Stopwatch stopwatch = Stopwatch.StartNew();

        await next(
            context);

        stopwatch.Stop();

        Console.WriteLine(
            stopwatch.ElapsedMilliseconds);
    });
```

This naturally measures:

EXAMPLE / FLOW

```
everything downstream
```

because the stopwatch starts before calling `next` and stops after the response returns.

This pattern is useful for:

```
Timing

Tracing

Logging

Response headers

Metrics
```

29. Example Logging Middleware

C#

```
public sealed class RequestLoggingMiddleware
{
    private readonly RequestDelegate _next;

    private readonly ILogger<
        RequestLoggingMiddleware>
        _logger;

    public RequestLoggingMiddleware(
        RequestDelegate next,
        ILogger<
            RequestLoggingMiddleware>
            logger)
    {
        _next = next;

        _logger = logger;
    }

    public async Task InvokeAsync(
        HttpContext context)
    {
        Stopwatch stopwatch = Stopwatch.StartNew();

        try
        {
            await _next(
                context);
        }
        finally
        {
            stopwatch.Stop();

            _logger.LogInformation(
                "HTTP {Method} {Path} returned {StatusCode} in {ElapsedMilliseconds} ms",
                context.Request.Method,
                context.Request.Path,
                context.Response.StatusCode,
                stopwatch.ElapsedMilliseconds);
        }
    }
}
```

30. Register Custom Middleware

Create an extension:

```C#
public static class
    RequestLoggingMiddlewareExtensions
{
    public static IApplicationBuilder
        UseRequestLogging(
            this IApplicationBuilder app)
    {
        return app
            .UseMiddleware<
                RequestLoggingMiddleware>();
    }
}
```

Then:

```C#
app.UseRequestLogging();
```

Microsoft recommends encapsulating custom middleware in a class and usually exposing it through an extension method.

PART 3 — CUSTOM MIDDLEWARE LIFETIMES

31. Conventional Middleware Is Constructed Once

A convention-based middleware class:

```C#
public sealed class TenantMiddleware
{
    private readonly RequestDelegate _next;

    public TenantMiddleware(
        RequestDelegate next)
    {
        _next = next;
    }

    public Task InvokeAsync(
        HttpContext context)
    {
        return _next(
            context);
    }
}
```

is generally created when the application pipeline is built.

Therefore constructor-injected state behaves as application-long middleware state.

Microsoft's middleware guidance states that convention-based middleware is constructed once for the application's lifetime.

32. Do Not Inject Scoped Service Into Conventional Middleware Constructor

Bad:

```C#
public sealed class TenantMiddleware
{
    private readonly
        FundraisingDbContext
        _dbContext;

    public TenantMiddleware(
        RequestDelegate next,
        FundraisingDbContext dbContext)
    {
        _dbContext = dbContext;
    }
}
```

`FundraisingDbContext` is scoped.

The conventional middleware itself is long-lived.

That creates incorrect lifetime semantics.

33. Inject Scoped Dependency Into InvokeAsync

Instead:

```C#
public sealed class TenantMiddleware
{
    private readonly RequestDelegate _next;

    public TenantMiddleware(
        RequestDelegate next)
    {
        _next = next;
    }

    public async Task InvokeAsync(
        HttpContext context,
        ITenantContext tenantContext)
    {
        // Use scoped service.

        await _next(
            context);
    }
}
```

ASP.NET Core can resolve additional `InvokeAsync` parameters from DI per request. Microsoft explicitly recommends this technique for scoped dependencies in conventional middleware.

34. IMiddleware

An alternative is implementing:

```C#
IMiddleware
```

Example:

```C#
public sealed class TenantMiddleware :
    IMiddleware
{
    private readonly ITenantContext _tenantContext;

    public TenantMiddleware(
        ITenantContext tenantContext)
    {
        _tenantContext = tenantContext;
    }

    public async Task InvokeAsync(
        HttpContext context,
        RequestDelegate next)
    {
        await next(
            context);
    }
}
```

Registration:

```C#
builder.Services
    .AddScoped<
        TenantMiddleware>();

app.UseMiddleware<
    TenantMiddleware>();
```

Factory-activated `IMiddleware` supports per-request activation and therefore allows scoped constructor dependencies.

35. Conventional Middleware vs IMiddleware

Feature	Conventional	IMiddleware
Creation	Pipeline/application construction	Per request through factory
Scoped constructor dependency	No	Yes
`RequestDelegate`	Constructor	`InvokeAsync` parameter
Registration in DI	Not always required explicitly	Required
Simple middleware	Excellent	Also possible
Per-request activation needed	Use `InvokeAsync` injection	Natural

PART 4 — ROUTING FUNDAMENTALS

36. What Is Routing?

Routing matches an incoming HTTP request to an executable endpoint.

Example:

EXAMPLE / FLOW

```
GET /api/gifts/125
```

might match:

C#

```
[HttpGet(
    "{giftId:long}")]
public async Task<
    ActionResult<GiftResponse>>
    Get(
        long giftId)
```

Routing determines:

EXAMPLE / FLOW

```
Which endpoint?

Which route values?

Which metadata?

Which HTTP method?

Which constraints?
```

ASP.NET Core routing performs URL-template matching, applies route constraints and other matcher policies, then selects the best endpoint.

37. What Is an Endpoint?

An endpoint represents executable request-handling logic plus metadata.

Examples:

EXAMPLE / FLOW

```
Controller action

Minimal API handler

Razor Page

SignalR hub endpoint
```

An endpoint can contain metadata describing:

EXAMPLE / FLOW

```
Authorization

HTTP method

Route name

CORS

Content types

Custom metadata
```

Routing selects an endpoint.

Later middleware and framework components can inspect its metadata.

38. Endpoint Routing Mental Model

EXAMPLE / FLOW

```
Incoming request
      ↓
Path matching
      ↓
Route constraints
      ↓
Matcher policies
      ↓
Endpoint selection
      ↓
Selected Endpoint
      ↓
Authorization / endpoint-aware middleware
      ↓
Endpoint execution
```

Microsoft documents matching in these phases: template matching, constraint filtering, matcher-policy filtering, and final endpoint selection.

39. Basic Minimal API Route

C#

```
app.MapGet(
    "/api/gifts/{giftId}",
    (
        long giftId) =>
    {
        return Results.Ok(
            giftId);
    });
```

Request:

EXAMPLE / FLOW

```
GET /api/gifts/100
```

route value:

EXAMPLE / FLOW

```
giftId = 100
```

40. Controller Attribute Routing

Example:

C#

```
[ApiController]
[Route(
    "api/gifts")]
public sealed class
    GiftsController :
    ControllerBase
{
    [HttpGet(
        "{giftId:long}")]
    public IActionResult Get(
```

```
        long giftId)
    {
        return Ok(
            giftId);
    }
}
```

PIPELINE / ROUTING

Then:

```
C#
app.MapControllers();
```

maps attribute-routed controllers.

Microsoft documents `MapControllers` for attribute-routed controllers and `MapControllerRoute` / `MapAreaControllerRoute` for conventional controller routing.

41. Route Template Components

Route:

```
EXAMPLE / FLOW
/api/gifts/{giftId:long}
```

contains:

```
EXAMPLE / FLOW
api
    →
literal segment

gifts
    →
literal segment

{giftId:long}
    →
route parameter + constraint
```

42. Literal Route Segments

Example:

```
EXAMPLE / FLOW
/api/gifts
```

`api` and `gifts` must match literally.

Request:

```
EXAMPLE / FLOW
/api/gifts
```

matches.

Request:

```
EXAMPLE / FLOW
/api/campaigns
```

does not.

43. Route Parameters

Example:

```
C#
app.MapGet(
    "/api/gifts/{giftId}",
    (
        long giftId) =>
        giftId);
```

Requests:

```
EXAMPLE / FLOW
/api/gifts/10

/api/gifts/20
```

produce:

```
EXAMPLE / FLOW
giftId = 10

giftId = 20
```

44. Optional Parameters

Template:

```
EXAMPLE / FLOW
/api/gifts/{giftId?}
```

means the parameter may be absent.

Use optional routing carefully in APIs because separate explicit routes are often easier to understand.

45. Default Route Values

A route may provide defaults.

Traditional conventional routing example:

```
C#
app.MapControllerRoute(
    name:
        "default",
    pattern:
        "{controller=Home}/{action=Index}/{id?}");
```

Defaults:

```
EXAMPLE / FLOW
controller = Home

action = Index
```

Microsoft documents this pattern as the conventional default controller route.

46. Catch-All Parameters

Example:

```
EXAMPLE / FLOW
/files/{*path}
```

could match:

```
EXAMPLE / FLOW
/files/reports/2026/october/report.pdf
```

with:

```
EXAMPLE / FLOW
path =
reports/2026/october/report.pdf
```

Catch-all routes are intentionally broad and normally have low route precedence.

47. Route Constraints

Example:

```
EXAMPLE / FLOW
/api/gifts/{giftId:long}
```

The:

```
EXAMPLE / FLOW
:long
```

constraint means the route candidate requires a value compatible with a long parameter.

Another route:

```
EXAMPLE / FLOW
/api/gifts/recent
```

can coexist with parameterized routes.

48. Common Constraint Examples

Examples include:

```
EXAMPLE / FLOW
{id:int}

{id:long}

{name:alpha}

{id:guid}

{value:bool}

{date:datetime}
```

```
{name:minlength(3)}
```

Constraints help routing distinguish among candidate endpoints.

49. Route Constraints Are Not Input Validation

This is a very important interview concept.

Suppose:

EXAMPLE / FLOW

```
/api/gifts/{giftId:int}
```

and the client sends:

EXAMPLE / FLOW

```
/api/gifts/abc
```

The route doesn't match, which can produce:

EXAMPLE / FLOW

```
404 Not Found
```

If `"abc"` is considered invalid user input that should produce a validation response:

EXAMPLE / FLOW

```
400 Bad Request
```

then route constraints should not be used as the validation mechanism.

Microsoft explicitly warns that route constraints are intended for route disambiguation, not input validation.

50. Route Specificity

Consider:

EXAMPLE / FLOW

```
/api/gifts/recent

/api/gifts/{giftId}
```

Request:

EXAMPLE / FLOW

```
/api/gifts/recent
```

could technically match both.

Routing prefers the more specific literal route:

EXAMPLE / FLOW

```
/api/gifts/recent
```

Route-template precedence favors more specific routes such as literal segments over unconstrained parameters.

51. Route Precedence Rules

Broadly:

EXAMPLE / FLOW

```
More specific
    ↓

Literal segment

Constrained parameter

Parameter

Catch-all

    ↓
Less specific
```

For example:

EXAMPLE / FLOW

```
/api/gifts/list
```

has higher specificity than:

EXAMPLE / FLOW

```
/api/gifts/{value}
```

A constrained parameter such as:

EXAMPLE / FLOW

```
{id:int}
```

is more specific than:

EXAMPLE / FLOW

```
{id}
```

Microsoft documents these precedence concepts so common route definitions don't need manual ordering.

52. Ambiguous Routes

Suppose two endpoints are effectively equally eligible:

EXAMPLE / FLOW

```
/api/gifts/{id}

/api/gifts/{value}
```

for:

EXAMPLE / FLOW

```
/api/gifts/100
```

The runtime may be unable to select one uniquely.

The final endpoint-selection phase throws an ambiguous-match exception when multiple candidates have the same highest priority.

Avoid designing ambiguous endpoint sets.

53. HTTP Method Matching

These are different endpoints:

C#
```
app.MapGet(
    "/api/gifts",
    GetGifts);

app.MapPost(
    "/api/gifts",
    CreateGift);
```

Same path:

EXAMPLE / FLOW
```
/api/gifts
```

Different verbs:

EXAMPLE / FLOW
```
GET
POST
```

Routing/matcher policies include HTTP method information when selecting the endpoint.

54. Attribute Routing in Web APIs

A typical API controller:

C#
```
[ApiController]
[Route(
    "api/[controller]")]
public sealed class
    GiftsController :
    ControllerBase
{
    [HttpGet]
    public IActionResult GetAll()
    {
        return Ok();
    }

    [HttpGet(
        "{giftId:long}")]
    public IActionResult Get(
        long giftId)
    {
        return Ok();
    }

    [HttpPost]
    public IActionResult Create()
    {
        return Ok();
    }
}
```

Routes:

EXAMPLE / FLOW

```
GET  /api/gifts

GET  /api/gifts/123

POST /api/gifts
```

55. Route Tokens

Controller routing supports tokens such as:

EXAMPLE / FLOW

```
[controller]

[action]

[area]
```

For:

C#

```
GiftsController
```

the token:

EXAMPLE / FLOW

```
[controller]
```

becomes roughly:

EXAMPLE / FLOW

```
Gifts
```

This can reduce duplicated route text.

56. Prefer Stable Resource-Oriented API Routes

For REST-style APIs, prefer routes like:

EXAMPLE / FLOW

```
GET    /api/gifts/100

POST   /api/gifts

PUT    /api/gifts/100

DELETE /api/gifts/100
```

rather than RPC-like paths everywhere:

EXAMPLE / FLOW

```
/api/GetGiftById

/api/CreateGift

/api/DeleteGift
```

Action-style endpoints can still be appropriate for domain operations such as:

EXAMPLE / FLOW

```
POST /api/gifts/100/complete
```

because:

EXAMPLE / FLOW

```
complete
```

represents an explicit business transition.

57. Route Groups

Minimal APIs can organize routes with:

C#

```
RouteGroupBuilder gifts =
    app.MapGroup(
        "/api/gifts");
```

Then:

C#

```
gifts.MapGet(
    "/{giftId:long}",
    GetGift);

gifts.MapPost(
    "/",
    CreateGift);

gifts.MapPost(
    "/{giftId:long}/complete",
    CompleteGift);
```

This avoids repeatedly writing the same prefix and provides a convenient place to apply group metadata/conventions.

58. Route Metadata

Metadata may include things such as:

EXAMPLE / FLOW

```
Authorization policy

CORS policy

Endpoint name

Rate limiting

Custom metadata
```

Middleware after route selection can inspect:

C#

```
Endpoint? endpoint = context.GetEndpoint();
```

and examine its metadata.

This is one reason routing must sometimes occur before middleware that depend information.

59. UseRouting in Modern WebApplication

With the modern `WebApplication` model, applications often don't need to explicitly call:

```
C#
app.UseRouting();

app.UseEndpoints(...);
```

for simple cases.

The framework can arrange routing around mapped endpoints.

Microsoft's current controller routing guidance notes that `WebApplicationBuilder` typically configures routing/endpoints around middleware automatically.

However, explicit:

```
C#
app.UseRouting();
```

is useful when you need to carefully position middleware **after endpoint selection but before endpoint execution**.

60. Endpoint-Aware Middleware

Example:

```
C#
app.UseRouting();

app.Use(
    async (
        context,
        next) =>
    {
        Endpoint? endpoint = context.GetEndpoint();

        if (
            endpoint is not null)
        {
            // Inspect endpoint metadata.
        }

        await next(
            context);
    });

app.UseAuthentication();

app.UseAuthorization();

app.MapControllers();
```

After routing:

is available.

Before routing:


```
context.GetEndpoint()
```

may not yet contain the selected endpoint.

61. ShortCircuit Endpoint Support

Current ASP.NET Core routing can explicitly mark certain endpoints to short-circuit later middleware.

Example:

C#

```
app.MapGet(
        "/robots.txt",
        () =>
            "User-agent: *")
    .ShortCircuit();
```

This tells routing that once the endpoint matches, it can execute immediately rather than running the rest of the normal downstream middleware.

Microsoft documents `ShortCircuit` and `MapShortCircuit` for known endpoints that don't need the full later pipeline.

Use carefully:

> ⓘ A short-circuited endpoint can intentionally bypass middleware such as authentication or CORS, so only use it where that behavior is safe.

PART 5 — ERROR HANDLING

62. Why Centralized Error Handling Matters

Without centralized handling:

C#

```
[HttpGet(
    "{giftId}")]
public async Task<IActionResult> Get(
    long giftId)
{
    try
    {
        // Work...
    }
    catch (
        GiftNotFoundException)
    {
        return NotFound();
    }
    catch (
        Exception)
```

```
    {
        return StatusCode(
            500);
    }
}
```


Every endpoint repeats:

```
try/catch
logging
status mapping
response formatting
```

This creates inconsistency.

Instead:

```
Controller / Endpoint
    ↓ throws meaningful exception
Central error handling
    ↓
HTTP response
```

63. Expected Error vs Unexpected Exception

Distinguish:

```
Expected business/API condition
```

from:

```
Unexpected system failure
```

Examples of expected conditions:

```
Gift not found

Validation failure

Concurrency conflict

Unauthorized operation
```

Examples of unexpected failures:

```
NullReferenceException

Database driver bug

Unexpected configuration failure

Coding defect
```

These should not necessarily produce identical HTTP responses.

64. Developer Exception Page

In Development, ASP.NET Core can provide detailed developer diagnostics for unhandled exceptions.

It can include:

EXAMPLE / FLOW

```
Stack trace

Request headers

Cookies

Query data

Endpoint metadata
```

The Developer Exception Page should only be exposed in Development because detailed internal information is a security risk in production. Current templates enable developer exception behavior automatically in appropriate Development scenarios.

65. Never Expose Stack Traces in Production

Bad production response:

C#

```
{
  "error":
    "System.InvalidOperationException at GiftRepository.cs line 82..."
}
```

This may reveal:

EXAMPLE / FLOW

```
Source paths

Database details

Class names

Internal framework details

Implementation architecture

Potential security-sensitive information
```

Clients usually need:

EXAMPLE / FLOW

```
status
error type
safe message
trace/correlation identifier
```

while detailed diagnostics belong in server logs.

66. UseExceptionHandler

In production, central exception handling can be configured with:

```C#
app.UseExceptionHandler();
```

when appropriate fallback handling such as Problem Details is registered.

Or:

```C#
app.UseExceptionHandler(
    "/error");
```

or a custom handler pipeline.

The exception handler middleware catches unhandled exceptions from downstream middleware and produces controlled error responses.

67. Problem Details

HTTP APIs commonly represent errors using:

```EXAMPLE / FLOW
Problem Details
```

A response may look like:

```C#
{
  "type": "https://example.com/problems/gift-not-found",
  "title": "Gift was not found.",
  "status": 404,
  "detail": "Gift 125 does not exist.",
  "instance": "/api/gifts/125",
  "traceId": "..."
}
```

ASP.NET Core provides:

```C#
IProblemDetailsService
```

and:

```C#
builder.Services
    .AddProblemDetails();
```

to support centralized Problem Details generation.

68. Basic Problem Details Configuration

```C#
var builder =
    WebApplication.CreateBuilder(
        args);

builder.Services
    .AddControllers();

builder.Services
```

```
    .AddProblemDetails();

var app = builder.Build();

app.UseExceptionHandler();

app.UseStatusCodePages();

app.MapControllers();

app.Run();
```

With `AddProblemDetails`, the built-in exception/status-code middleware can generate Problem Details responses where appropriate.

69. IExceptionHandler

Modern ASP.NET Core provides:

```
C#
IExceptionHandler
```

for centralized exception-handling logic.

Interface conceptually:

```
C#
public interface IExceptionHandler
{
    ValueTask<bool>
        TryHandleAsync(
            HttpContext httpContext,
            Exception exception,
            CancellationToken cancellationToken);
}
```

Return:

```
EXAMPLE / FLOW
true
    →
exception handled

false
    →
allow next exception handler/fallback behavior
```

Multiple handlers can be registered and are invoked in registration order until one handles the exception.

70. Register IExceptionHandler

Example:

```
C#
builder.Services
    .AddExceptionHandler<
        GiftNotFoundExceptionHandler>();

builder.Services
    .AddProblemDetails();
```

Pipeline:


```
C#
app.UseExceptionHandler();
```

Registering the handler alone is not enough—the exception-handling middleware must also be present.

71. GiftNotFoundExceptionHandler

```
C#
public sealed class
    GiftNotFoundExceptionHandler :
    IExceptionHandler
{
    private readonly
        IProblemDetailsService
        _problemDetails;

    public GiftNotFoundExceptionHandler(
        IProblemDetailsService problemDetails)
    {
        _problemDetails = problemDetails;
    }

    public async ValueTask<bool>
        TryHandleAsync(
            HttpContext context,
            Exception exception,
            CancellationToken cancellationToken)
    {
        if (
            exception
            is not
            GiftNotFoundException
            giftNotFound)
        {
            return false;
        }

        context.Response.StatusCode = StatusCodes.Status404NotFound;

        return await _problemDetails
            .TryWriteAsync(
                new ProblemDetailsContext
                {
                    HttpContext =
                        context,

                    ProblemDetails =
                        new ProblemDetails
                        {
                            Status =
                                StatusCodes
                                    .Status404NotFound,

                            Title =
                                "Gift was not found.",

                            Detail =
                                $"Gift {giftNotFound.GiftId} was not found."
                        }
                });
```

```
    }
}
```


If an `IExceptionHandler` returns `true`, it should write a complete response including an appropriate status code.

72. Exception Handler Ordering

You can register several handlers:

```C#
builder.Services
    .AddExceptionHandler<
        GiftNotFoundExceptionHandler>();

builder.Services
    .AddExceptionHandler<
        ConcurrencyExceptionHandler>();

builder.Services
    .AddExceptionHandler<
        GeneralExceptionHandler>();
```

Conceptually:

```EXAMPLE / FLOW
Exception
    ↓
GiftNotFound handler
    ↓ not handled?
Concurrency handler
    ↓ not handled?
General handler
```

Handlers are invoked in registration order.

Put:

```EXAMPLE / FLOW
specific handlers
```

before:

```EXAMPLE / FLOW
broad fallback handlers
```

if the broad handler would otherwise consume every exception.

73. Important .NET 10 Exception Diagnostics Change

This is a current ASP.NET Core 10 behavior worth remembering.

Starting in **.NET 10**, when an `IExceptionHandler` handles an exception by returning:

```EXAMPLE / FLOW
true
```

the exception-handling middleware suppresses its normal diagnostics such as logs and metrics for that handled exception by default.

Earlier .NET 8/9 behavior emitted those diagnostics even when the exception was handled.

This can be customized using:


```
C#
ExceptionHandlerOptions
    .SuppressDiagnosticsCallback
```

74. Why This Matters

Suppose your custom handler logs:

```
C#
_logger.LogWarning(
    exception,
    "Gift was not found.");
```

In .NET 10:

```
EXAMPLE / FLOW
handled exception
    ↓
middleware diagnostics suppressed by default
```

which can prevent duplicate diagnostics.

If your observability strategy requires middleware diagnostics anyway, configure:

```
C#
app.UseExceptionHandler(
    new ExceptionHandlerOptions
    {
        SuppressDiagnosticsCallback =
            _ =>
                false
    });
```

Use this deliberately rather than memorizing old .NET 8/9 behavior.

75. Exception-to-HTTP Mapping

A possible Fundraising mapping might be:

Exception/condition	HTTP
`GiftNotFoundException`	404
Validation problem	400
Unauthorized/unauthenticated	401
Authenticated but forbidden	403
Concurrency conflict	409
Duplicate external reference	409
Unexpected exception	500

The exact mapping should reflect your API contract.

Do not map every:

C#

```
InvalidOperationException
```

PIPELINE / ROUTING

to:

EXAMPLE / FLOW

```
400
```

because `InvalidOperationException` may also represent programmer/system errors.

Prefer meaningful application/domain exception types where HTTP mapping is required.

76. 400 vs 404

Example:

EXAMPLE / FLOW

```
GET /api/gifts/abc
```

If routing requires:

EXAMPLE / FLOW

```
{giftId:long}
```

the route may not match and produce:

EXAMPLE / FLOW

```
404
```

But if the endpoint intentionally accepts a string and validates it:

EXAMPLE / FLOW

```
"abc is not a valid gift identifier"
```

then:

EXAMPLE / FLOW

```
400
```

may be appropriate.

This is exactly why Microsoft warns against using route constraints as general input validation.

77. 401 vs 403

Although security is covered deeply in Topic 40:

EXAMPLE / FLOW

```
401 Unauthorized
```

normally means:

EXAMPLE / FLOW

```
the caller is not successfully authenticated
```

while:

normally means:

```
EXAMPLE / FLOW
caller is authenticated
but lacks required permission
```

Do not return:

```
EXAMPLE / FLOW
500
```

for ordinary authorization failures.

78. 409 Conflict

Useful for certain conflicts such as:

```
EXAMPLE / FLOW
Optimistic concurrency failure

Duplicate business identifier

State conflict
```

Example:

```
EXAMPLE / FLOW
Gift already completed
```

might be:

```
EXAMPLE / FLOW
409 Conflict
```

depending on your API contract.

The important part is consistency.

79. 500 Internal Server Error

Use:

```
EXAMPLE / FLOW
500
```

for unexpected server failures.

The client should normally receive a generic safe response such as:

```
C#
{
  "title": "An unexpected error occurred.",
  "status": 500,
  "traceId": "..."
}
```

while server logs contain detailed exception information.

80. ProblemDetails Customization

You can customize globally:

```C#
builder.Services
    .AddProblemDetails(
        options =>
        {
            options.CustomizeProblemDetails =
                context =>
                {
                    context.ProblemDetails
                        .Extensions[
                            "traceId"] =
                        context.HttpContext
                            .TraceIdentifier;
                };
        });
```

This can standardize fields such as:

```EXAMPLE / FLOW
traceId

errorCode

documentation type URI
```

Do not put sensitive exception internals into the response.

81. Status Code Pages

Exception handling handles:

```EXAMPLE / FLOW
exceptions
```

but not every error response is caused by an exception.

For example:

```EXAMPLE / FLOW
404 route not found
```

can happen with no exception.

`UseStatusCodePages` can generate response content for error status codes that otherwise have no body.

Example:

```C#
app.UseStatusCodePages();
```

When combined with:

```C#
builder.Services
    .AddProblemDetails();
```

the middleware can generate Problem Details for suitable empty error responses.

82. Exception vs Status Code

Understand the difference:

```EXAMPLE / FLOW
throw new GiftNotFoundException()
    ↓
Exception handler
```

versus:

```C#
return Results.NotFound();
```

or:

```C#
context.Response.StatusCode = 404;
```

which is simply an HTTP status response.

No exception necessarily occurred.

83. Don't Throw Exceptions for Every Expected Validation Failure

Bad:

```C#
foreach (
    ValidationError error
    in errors)
{
    throw new Exception(
        error.Message);
}
```

Routine request validation is often better represented directly as a validation result.

Exceptions are particularly valuable when:

```EXAMPLE / FLOW
normal control flow cannot continue
```

but they should not become a replacement for every conditional branch.

84. Exception Middleware Cannot Rewrite Started Response

Suppose endpoint does:

```C#
await context.Response
    .WriteAsync(
        "Beginning response...");

throw new Exception();
```

The response may already be sent partially.

The exception handler cannot simply replace everything with a clean JSON error response.

Microsoft documents that once headers are sent, status/headers can't be changed and normal exception pages/handlers can't restart response generation.

Therefore:

> Avoid writing partial responses before operations likely to fail unless streaming semantics explicitly require it.

85. Error Handler Must Be Reliable

Ironically, your error handler can also throw.

Bad:

```C#
catch (
    Exception exception)
{
    await unreliableRemoteLogger
        .SendAsync(
            exception);

    // Remote logging fails and throws again.
}
```

Production exception-handling code should be:

```EXAMPLE / FLOW
small
defensive
well tested
```

Microsoft explicitly recommends testing production error handlers carefully because error-handling code itself can fail.

PART 6 — COMPLETE FUNDRAISING PIPELINE

86. Program.cs Example

```C#
var builder =
    WebApplication.CreateBuilder(
        args);

builder.Services
    .AddControllers();
```

```
builder.Services
    .AddProblemDetails();

builder.Services
    .AddExceptionHandler<
        GiftNotFoundExceptionHandler>();

builder.Services
    .AddExceptionHandler<
        GiftConcurrencyExceptionHandler>();

builder.Services
    .AddFundraisingApplication();

builder.Services
    .AddFundraisingInfrastructure(
        builder.Configuration);

builder.Services
    .AddAuthentication();

builder.Services
    .AddAuthorization();

var app = builder.Build();

app.UseExceptionHandler();

app.UseStatusCodePages();

app.UseHttpsRedirection();

app.UseRouting();

app.UseRequestLogging();

app.UseAuthentication();

app.UseAuthorization();

app.MapControllers();

app.Run();
```

87. Request Example

Request:

EXAMPLE / FLOW

```
POST /api/gifts/125/complete
```

Flow:

EXAMPLE / FLOW

```
Kestrel
   ↓
Exception Handler
   ↓
HTTPS Redirection
   ↓
Routing
   ↓
Request Logging
```

```
   ↓
Authentication
   ↓
Authorization
   ↓
GiftsController.Complete
   ↓
CompleteGiftService
   ↓
GiftRepository
   ↓
Gift.Complete()
   ↓
SaveChanges
   ↓
Controller result
   ↓
Response travels back outward
```

Diagram 2 — Full Request Pipeline

EXAMPLE / FLOW

```
CLIENT
  │
  ▼
Kestrel
  │
  ▼
ExceptionHandler
  │
  ▼
HTTPS Redirection
  │
  ▼
Routing
  │
  ├── Selects:
  │   POST /api/gifts/{giftId}/complete
  │
  ▼
Request Logging
  │
  ▼
Authentication
  │
  ▼
Authorization
  │
  ▼
Controller Endpoint
  │
  ▼
CompleteGiftService
  │
  ▼
Domain + EF Core
  │
  ▼
HTTP Result
  │
  ▼
Authorization
  │
```

```
  ▼
Authentication
  |
  ▼
Request Logging
  |
  ▼
Exception Handler
  |
  ▼
Kestrel
  |
  ▼
CLIENT
```

88. Successful Request

Suppose:

EXAMPLE / FLOW

```
Gift 125 exists

Caller authorized

Gift valid for completion
```

Controller:

C#

```
[HttpPost(
    "{giftId:long}/complete")]
public async Task<IActionResult>
    Complete(
        long giftId,
        CancellationToken cancellationToken)
{
    await _service
        .CompleteAsync(
            new CompleteGiftCommand(
                _tenant.TenantId,
                giftId),
            cancellationToken);

    return NoContent();
}
```

Result:

EXAMPLE / FLOW

```
204 No Content
```

89. Gift Not Found

Application throws:

C#

```
throw new
    GiftNotFoundException(
        giftId);
```

Stack unwinds:

EXAMPLE / FLOW

```
Application Service
    ↑
Controller
    ↑
Authorization
    ↑
Authentication
    ↑
Request Logging
    ↑
Exception Handler
```

`GiftNotFoundExceptionHandler` maps to:

EXAMPLE / FLOW

```
404 Not Found
```

Problem Details response:

C#

```
{
  "title": "Gift was not found.",
  "status": 404,
  "detail": "Gift 125 was not found.",
  "traceId": "..."
}
```

90. Unexpected Failure

Suppose SQL Server driver unexpectedly throws.

No specific handler handles it.

Fallback handling returns:

EXAMPLE / FLOW

```
500 Internal Server Error
```

Client sees:

C#

```
{
  "title": "An unexpected error occurred.",
  "status": 500,
  "traceId": "..."
}
```

Logs contain the technical details.

Client does not receive the internal stack trace.

PART 7 — ROUTING DESIGN FOR THE FUNDRAISING API

91. Gift Endpoints

A coherent set might be:

EXAMPLE / FLOW

```
GET    /api/gifts
GET    /api/gifts/{giftId}
POST   /api/gifts
POST   /api/gifts/{giftId}/complete
POST   /api/gifts/{giftId}/cancel
GET    /api/gifts/{giftId}/allocations
```

92. Campaign Endpoints

EXAMPLE / FLOW

```
GET    /api/campaigns
GET    /api/campaigns/{campaignId}
POST   /api/campaigns
PUT    /api/campaigns/{campaignId}
```

93. Fund Endpoints

EXAMPLE / FLOW

```
GET    /api/funds
GET    /api/funds/{fundId}
POST   /api/funds
```

94. Nested Resources

Possible:

EXAMPLE / FLOW

```
/api/gifts/{giftId}/allocations
```

because allocations naturally belong to a Gift.

Be careful with deeply nested routes:

EXAMPLE / FLOW

```
/api/tenants/1/constituents/2/gifts/3/allocations/4/fund/5
```

Deep nesting becomes difficult to maintain.

Tenant identity may also come from authenticated context rather than public route parameters depending on your security model.

95. Route IDs and Tenant Security

This route:

EXAMPLE / FLOW

```
/api/gifts/125
```

does not prove the caller can access Gift 125.

Routing answers:

EXAMPLE / FLOW

```
Which endpoint?
```

Authorization/application/data filtering answers:

EXAMPLE / FLOW

```
May this caller access this Gift?
```

Never confuse route matching with access control.

PART 8 — PERFORMANCE

96. Every Middleware Adds Work

Every request may pass through:

EXAMPLE / FLOW

```
Logging

Tracing

Authentication

Authorization

Rate limiting

CORS

Custom validation

Metrics
```

Each component adds some cost.

Usually this overhead is small relative to:

EXAMPLE / FLOW

```
SQL
Network
Serialization
```

but unnecessary middleware should still be avoided.

97. Put Cheap Rejections Early When Safe

If a request can be safely rejected before expensive processing:

EXAMPLE / FLOW

```
Request
    ↓
cheap check
    ↓ reject
```

you save:

EXAMPLE / FLOW

```
database work
network calls
serialization
```

But security-sensitive middleware must still respect correct ordering.

Optimization must not bypass required authentication or authorization.

PIPELINE / ROUTING

98. Avoid Expensive Work Before Routing Unless Every Request Needs It

Suppose middleware calls SQL Server before routing:

EXAMPLE / FLOW

```
Every request
    ↓
database query
    ↓
routing
```

Even:

EXAMPLE / FLOW

```
/favicon.ico
```

may perform the database call.

If the work applies only to selected endpoints, use:

EXAMPLE / FLOW

```
endpoint metadata

authorization policy

endpoint filter

application service
```

or place middleware after routing as appropriate.

99. Avoid Reading Large Request Body in Generic Middleware

Reading request body can:

EXAMPLE / FLOW

```
allocate memory

consume streams

require buffering

increase latency
```

If the body must be read again downstream, buffering or caching may be required.

Do not implement generic request-body logging that blindly loads multi-megabyte payloads into memory.

100. Routing Performance

Routing is optimized around endpoint matching and precedence.

Do not manually implement giant routing switches:

```C#
if (
    path.StartsWith(
        "/api/gifts"))
{
}
else if (...)
{
}
```

when endpoint routing already solves the problem.

PART 9 — SECURITY

101. Middleware Order Can Be a Security Boundary

Incorrect:

```EXAMPLE / FLOW
Authorization
    ↓
Authentication
```

Authorization may not see the correct authenticated identity.

Correct:

```EXAMPLE / FLOW
Authentication
    ↓
Authorization
```

Middleware ordering is therefore not just style—it affects security.

102. CORS Ordering Matters

When CORS is required, its middleware must be placed according to the documented pipeline ordering relative to routing/authentication/authorization.

Microsoft specifically identifies middleware ordering constraints around CORS, authentication, and authorization.

Topic 40 will cover CORS and security configuration more fully.

103. Don't Trust Route Values

This:

```EXAMPLE / FLOW
/api/tenants/10/gifts/125
```

does not mean the authenticated caller belongs to tenant 10.

Route values are client input.

Always verify tenant identity through trusted authorization/application boundaries.

104. Don't Log Sensitive Request Data

Middleware makes it tempting to log:

EXAMPLE / FLOW

```
Headers

Cookies

Query strings

Request bodies
```

These may contain:

EXAMPLE / FLOW

```
Authorization bearer tokens

Session cookies

PII

Payment information

Secrets
```

Use intentional redaction and allow-listing.

105. Production Error Responses Must Be Safe

Do not return:

EXAMPLE / FLOW

```
Connection string

SQL query

Stack trace

File path

User token

Internal exception data
```

through Problem Details.

Error handling is a security boundary.

PART 10 — TESTING

106. Unit Test Middleware?

Simple middleware logic can be unit tested by constructing a:

C#

```
DefaultHttpContext
```

and a test:

```
C#
RequestDelegate
```

Example:

```
C#
[Fact]
public async Task Middleware_CallsNext()
{
    bool called = false;

    RequestDelegate next =
        _ =>
        {
            called = true;

            return Task.CompletedTask;
        };

    RequestLoggingMiddleware middleware =
        new(
            next,
            NullLogger<
                RequestLoggingMiddleware>
                .Instance);

    DefaultHttpContext context = new();

    await middleware.InvokeAsync(
        context);

    Assert.True(
        called);
}
```

107. Integration Tests Are Often Better for Pipeline Behavior

Middleware order, routing, authentication, exception handling, and status-code formatting interact.

Therefore use:

```
C#
WebApplicationFactory<Program>
```

for important pipeline behavior.

Example tests:

```
EXAMPLE / FLOW
Unknown route → 404 ProblemDetails

Missing Gift → 404 ProblemDetails

Unhandled exception → 500 without stack trace

Unauthorized endpoint → expected auth response

Route /api/gifts/123 reaches correct endpoint
```

108. Routing Tests

Important routing tests may verify:

EXAMPLE / FLOW

```
Correct verb

Correct route parameter

Literal route beats parameter route

Invalid route doesn't accidentally hit another endpoint
```

You usually don't need to unit test ASP.NET Core's routing engine itself.

Test your route definitions.

109. Error Handler Tests

Given:

EXAMPLE / FLOW

```
GiftNotFoundException
```

verify:

EXAMPLE / FLOW

```
404

application/problem+json

expected title

trace identifier present

no stack trace
```

This is an ideal API integration test.

PART 11 — COMMON MISTAKES

110. Wrong Middleware Order

Bad:

C#

```
app.UseAuthorization();

app.UseAuthentication();
```

Authentication should normally precede authorization.

111. Exception Handler Too Late

Bad conceptual pipeline:

EXAMPLE / FLOW

```
Controller endpoint
    ↓
Exception handler
```

The handler cannot wrap failures that already occurred upstream.

Place global error handling early.

112. Calling next After Terminal Response

Bad:

C#

```
context.Response.StatusCode = 403;

await context.Response
    .WriteAsync(
        "Forbidden");

await next(
    context);
```

Once you've deliberately ended the request:

C#

```
return;
```

113. Using Route Constraint as Validation

Bad assumption:

EXAMPLE / FLOW

```
{id:int}
    =
input validation
```

A constraint is primarily routing disambiguation.

Invalid constrained values may cause 404 rather than the desired 400 response.

114. Catching Exception in Every Controller

Bad:

EXAMPLE / FLOW

```
50 endpoints
    ×
try/catch
```

Use centralized handling for cross-cutting exception translation.

115. Returning Exception.Message to Clients

Bad:

C#

```
return Problem(
    detail:
        exception.Message);
```

for arbitrary exceptions.

The message may contain internal information.

Map known safe failures deliberately.

116. Using Exception for Every 404

Sometimes:

```
C#
Gift? gift = await query.GetAsync(...);

return gift is null
    ? NotFound()
    : Ok(gift);
```

is perfectly reasonable.

You do not need an exception for every expected branch.

Exception-driven and result-driven styles can coexist.

117. Injecting Scoped DbContext Into Conventional Middleware Constructor

Conventional middleware lives across requests.

Inject scoped dependencies into:

```
EXAMPLE / FLOW
InvokeAsync
```

or use:

```
C#
IMiddleware
```

with per-request activation.

118. Giant Custom Middleware

Bad:

```
EXAMPLE / FLOW
Tenant resolution
Authentication
Authorization
Database transaction
Validation
Business rules
Logging
Exception handling
```

inside one middleware.

Middleware should have a focused cross-cutting/request-pipeline responsibility.

119. Business Logic in Middleware

Bad:

```C#
if (
    gift.TotalAmount
    >
    10_000m)
{
    gift.MarkAsMajorGift();
}
```

Middleware is an HTTP pipeline concern.

Gift business rules belong in the Domain/Application design.

120. Storing Per-Request Data in Singleton Middleware Fields

Bad:

```C#
public string? CurrentTenant
{
    get;
    set;
}
```

on conventional middleware.

Concurrent requests share the middleware instance.

Use:

EXAMPLE / FLOW

```
HttpContext

scoped service

Items
```

for request-specific information.

121. Writing Response Then Throwing

If possible, complete fallible work before sending response headers/body.

Once response output starts, centralized exception handling has limited ability to replace it.

122. Overusing Branching Middleware

Do not build your own routing system using:

```C#
if (
    context.Request.Path...)
```

Use endpoint routing for endpoint selection.

Middleware branching should solve middleware-level concerns.

PART 12 — BEST PRACTICES

Use middleware for cross-cutting HTTP concerns.

Keep middleware focused.

Understand forward request/reverse response execution.

Place exception handling early.

Call `next` exactly when downstream processing should continue.

Return without calling `next` when intentionally short-circuiting.

Do not call `next` after sending a terminal response.

Use `HttpContext.Items` only for appropriate per-request communication.

Avoid storing request-specific mutable state in middleware fields.

Respect DI lifetimes.

Inject scoped dependencies into `InvokeAsync` or use `IMiddleware`.

Use endpoint routing instead of custom path switches.

Use route constraints to disambiguate routes, not validate business input.

Prefer clear resource-oriented API routes.

Avoid ambiguous endpoint definitions.

Treat middleware order as a security and correctness concern.

Use centralized production exception handling.

Use `ProblemDetails` for consistent API errors where appropriate.

Do not expose stack traces or sensitive exception information.

Map known application/domain failures explicitly.

Keep unexpected failures as safe 500 responses.

Use Status Code Pages when empty error status responses need a standardized body.

Remember .NET 10's handled-exception diagnostics behavior.

Use integration tests for the complete pipeline.

PART 13 — PRACTICAL EXERCISES

Exercise 1 — Before and After Middleware

Create:

```C#
app.Use(
    async (
        context,
        next) =>
    {
        Console.WriteLine(
            "A before");

        await next(
            context);

        Console.WriteLine(
            "A after");
    });
```

Then add another middleware B.

Predict the output before running it.

Expected pattern:

EXAMPLE / FLOW

```
A before
B before
Endpoint
B after
A after
```

Exercise 2 — Short-Circuit

Create middleware that returns:

EXAMPLE / FLOW

```
503 Service Unavailable
```

for:

EXAMPLE / FLOW

```
/maintenance
```

without calling the next middleware.

Verify the controller is never reached.

Exercise 3 — Request Timing

Write:

EXAMPLE / FLOW

```
RequestTimingMiddleware
```

using `Stopwatch`.

Log:

EXAMPLE / FLOW

```
Method
Path
StatusCode
ElapsedMilliseconds
```

after calling `_next`.

Exercise 4 — Scoped Middleware Dependency

Create:

C#

```
ITenantContext
```

as scoped.

First attempt to use it incorrectly in conventional middleware constructor.

Then fix it by injecting into:

C#

```
InvokeAsync(...)
```

Exercise 5 — IMiddleware

Rewrite the Tenant middleware using:

```C#
IMiddleware
```

register it as scoped, and inject `ITenantContext` through the constructor.

Exercise 6 — Route Precedence

Create:

```EXAMPLE / FLOW
/api/gifts/recent

/api/gifts/{giftId}
```

Call:

```EXAMPLE / FLOW
/api/gifts/recent
```

and verify the literal route wins.

Exercise 7 — Route Constraint

Create:

```EXAMPLE / FLOW
/api/gifts/{giftId:long}
```

Test:

```EXAMPLE / FLOW
/api/gifts/100
/api/gifts/abc
```

Observe routing behavior.

Explain why the second case is route matching rather than business validation.

Exercise 8 — Centralized 404 Exception

Create:

```C#
GiftNotFoundException
```

and an:

```C#
IExceptionHandler
```

that produces a 404 Problem Details response.

Exercise 9 — Unexpected Exception

Create a test endpoint that throws:

```C#
InvalidOperationException
```

Verify production-style handling returns:

without exposing the stack trace.

Exercise 10 — Status Code Pages

Request an unknown route.

Configure:

```
C#
AddProblemDetails()

UseStatusCodePages()
```

and inspect the resulting error response.

Exercise 11 — Pipeline Ordering

Start with:

```
C#
app.UseAuthorization();

app.UseAuthentication();
```

then correct the order.

Explain why authentication must normally happen first.

Exercise 12 — API Integration Test

Using:

```
C#
WebApplicationFactory<Program>
```

verify:

```
EXAMPLE / FLOW
GET /api/gifts/999999
    →
404 ProblemDetails
```

and ensure the response doesn't contain internal stack-trace text.

PART 14 — EXACTLY 25 INTERVIEW QUESTIONS

1. What is middleware in ASP.NET Core?

Middleware is a component in the HTTP request-processing pipeline.

Each middleware can:

```
EXAMPLE / FLOW
inspect/modify request

perform work

invoke next middleware
```

```
inspect/modify response
*
short-circuit processing
```

PIPELINE / ROUTING

Middleware runs in registration order for requests and unwinds in reverse order for responses.

2. How does the ASP.NET Core request pipeline work?

Kestrel receives the request and passes an `HttpContext` through an ordered chain of middleware.

Conceptually:

EXAMPLE / FLOW

```
Request
    ↓
Middleware A
    ↓
Middleware B
    ↓
Endpoint
    ↑
Middleware B
    ↑
Middleware A
    ↑
Response
```

Code before `await next(...)` runs while the request travels inward; code after it runs while the response travels outward.

3. What is HttpContext?

`HttpContext` represents the current HTTP request and response.

It provides access to:

EXAMPLE / FLOW

```
Request

Response

User

RequestServices

Items

Features

Connection information

Trace identifier
```

It belongs to the current request and should not be treated as global mutable application state.

4. What is the difference between app.Use and app.Run?

`Use` adds middleware that may invoke the next component.

C#

```
app.Use(
    async (
        context,
```

```
        next) =>
    {
        await next(
            context);
    });
```

PIPELINE / ROUTING

`Run` adds terminal middleware with no `next` delegate.

The first terminal `Run` ends that pipeline.

5. What is middleware short-circuiting?

Short-circuiting means a middleware handles a request without invoking the next middleware.

Example:

EXAMPLE / FLOW

```
Request
    ↓
Maintenance middleware
    ↓
503
    ↓
STOP
```

It is useful for scenarios where downstream processing is unnecessary.

6. Why is middleware order important?

Middleware ordering controls:

EXAMPLE / FLOW

```
which component sees the request first

which component can wrap downstream exceptions

when authentication occurs

when endpoint metadata exists

when authorization executes

which work can be short-circuited
```

Incorrect ordering can cause security, performance, or functional defects. Microsoft explicitly describes middleware ordering as critical.

7. Why should authentication normally come before authorization?

Authentication establishes:

C#

```
HttpContext.User
```

and its identity/claims.

Authorization then uses that authenticated identity to decide whether the selected operation is allowed.

Conceptually:

EXAMPLE / FLOW

```
Authentication
    ↓
```

```
Who are you?
    *
Authorization
    ↓
Are you allowed?
```

PIPELINE / ROUTING

8. How do you create custom middleware?

Convention-based middleware usually contains:

EXAMPLE / FLOW

```
constructor(RequestDelegate next)

Invoke or InvokeAsync(HttpContext ...)
```

Example:

C#

```
public sealed class
    RequestTimingMiddleware
{
    private readonly RequestDelegate _next;

    public RequestTimingMiddleware(
        RequestDelegate next)
    {
        _next = next;
    }

    public async Task InvokeAsync(
        HttpContext context)
    {
        await _next(
            context);
    }
}
```

It is commonly registered with `UseMiddleware<T>()` and exposed through an extension method.

9. Can a conventional middleware constructor inject a scoped service?

Not normally.

Convention-based middleware is constructed once for the application pipeline, while scoped services belong to request scopes.

Inject scoped dependencies into:

C#

```
InvokeAsync(...)
```

or use factory-activated:

C#

```
IMiddleware
```

which supports per-request activation.

10. What is IMiddleware?

`IMiddleware` is an ASP.NET Core middleware abstraction that supports factory/DI-based activation.

Its main benefit is per-request activation, allowing scoped dependencies to be injected into the middleware constructor.

Implementations are registered with DI and resolved through `IMiddlewareFactory`.

11. What is routing in ASP.NET Core?

Routing matches an incoming request to an endpoint.

It considers:

```
EXAMPLE / FLOW
URL path

Route template

Route constraints

HTTP method

Matcher policies

Endpoint precedence
```

before final endpoint selection.

12. What is an endpoint?

An endpoint is executable request-handling logic plus associated metadata.

Examples include:

```
EXAMPLE / FLOW
Controller action

Minimal API handler

Razor Page
```

Endpoint metadata may describe authorization, HTTP method, CORS, rate limiting, and other policies.

13. What is a route template?

A route template describes the URL structure used to match an endpoint.

Example:

```
EXAMPLE / FLOW
/api/gifts/{giftId:long}
```

contains:

```
EXAMPLE / FLOW
api/gifts
    →
literal segments

giftId
    →
route parameter

long
```

```
    →
route constraint
```

14. What is a route constraint and what should it be used for?

A route constraint narrows which route values match a route.

Example:

EXAMPLE / FLOW

```
{id:int}
```

Constraints are primarily intended for route disambiguation.

They should not be used as ordinary request validation because an invalid constrained value commonly results in route failure and a 404 response instead of a validation-oriented 400.

15. How does ASP.NET Core choose between competing routes?

Routing first finds matching route templates, applies constraints and matcher policies, and then endpoint selection uses order and template precedence.

More-specific templates generally receive higher precedence.

For example:

EXAMPLE / FLOW

```
/api/gifts/recent
```

is more specific than:

EXAMPLE / FLOW

```
/api/gifts/{value}
```

for `/api/gifts/recent`.

Equal highest-priority candidates can produce an ambiguous-match exception.

16. What is the difference between MapControllers and MapControllerRoute?

`MapControllers()` maps attribute-routed controller endpoints.

`MapControllerRoute(...)` defines conventional controller routing and also participates in mapping applicable controller endpoints.

For Web APIs, attribute routing with `MapControllers()` is common.

17. Do modern ASP.NET Core applications always need explicit UseRouting and UseEndpoints?

No.

With the modern `WebApplication` hosting model, common applications can map endpoints directly and the framework arranges the routing/endpoint pipeline.

However, explicit `UseRouting()` is useful when middleware must deliberately execute after route selection and before endpoint execution, such as when it needs selected endpoint metadata.

18. What is the Developer Exception Page?

It is Development-only middleware that displays detailed information about unhandled request exceptions, potentially including:

EXAMPLE / FLOW

```
stack traces

headers

cookies

query data

endpoint metadata
```

It must not be exposed in production because that information can reveal sensitive implementation details.

19. What does UseExceptionHandler do?

`UseExceptionHandler` adds centralized exception-handling middleware.

It catches unhandled exceptions from downstream pipeline components and executes configured error-handling behavior such as:

EXAMPLE / FLOW

```
error endpoint

ProblemDetails generation

custom exception handlers
```

It should normally be positioned early enough to wrap downstream processing.

20. What is IExceptionHandler?

`IExceptionHandler` provides centralized typed/custom exception handling through:

C#

```
TryHandleAsync(
    HttpContext,
    Exception,
    CancellationToken)
```

Multiple implementations can be registered.

They execute in registration order until one returns `true`.

The exception handling middleware is still required for them to run.

21. What changed for handled-exception diagnostics in .NET 10?

Starting in .NET 10, when an `IExceptionHandler` returns `true`, exception-handling middleware suppresses its normal diagnostics such as logs and metrics for that handled exception by default.

Earlier .NET 8/9 behavior emitted middleware diagnostics even for handled exceptions.

The behavior can be customized through:

C#

```
SuppressDiagnosticsCallback
```

on `ExceptionHandlerOptions`.

22. What is ProblemDetails?

Problem Details is a standardized structure commonly used to represent HTTP API errors.

ASP.NET Core provides:

```
C#
IProblemDetailsService
```

and:

```
C#
services.AddProblemDetails();
```

to generate consistent problem responses from exception and status-code handling infrastructure.

23. What is the difference between exception handling and status code pages?

Exception handling deals with:

```
EXAMPLE / FLOW
thrown exceptions
```

while status-code pages handle error responses such as:

```
EXAMPLE / FLOW
404
400
403
```

that may exist without an exception and have no response body.

`UseStatusCodePages` can provide standardized bodies, including Problem Details when configured.

24. Why can't exception middleware always replace an error response after an exception?

Once the response has started:

```
EXAMPLE / FLOW
headers may already be sent

status code can no longer safely change

body may already be partially transmitted
```

Therefore centralized error middleware can't always replace a partially written response with a clean Problem Details document.

This is why fallible work should normally occur before response output begins unless streaming is intentional.

25. How would you design the Fundraising Platform request pipeline?

A practical starting pipeline is:

```
EXAMPLE / FLOW
Kestrel
    ↓
```

```
Global exception handling
    ↓
HTTPS handling
    ↓
Routing
    ↓
Request telemetry/logging
    ↓
CORS where required
    ↓
Authentication
    ↓
Authorization
    ↓
Selected Gift/Campaign/Fund endpoint
    ↓
Application service
    ↓
Domain
    ↓
Infrastructure
```

PIPELINE / ROUTING

Use:

EXAMPLE / FLOW

```
Middleware
    →
cross-cutting HTTP concerns

Routing
    →
endpoint selection

Authentication
    →
caller identity

Authorization
    →
access decision

Controller/Minimal API
    →
HTTP/application translation

Application Service
    →
use-case orchestration

Domain
    →
business rules

IExceptionHandler
    →
central exception → HTTP translation

ProblemDetails
    →
consistent safe error representation
```

For example:

EXAMPLE / FLOW

```
POST /api/gifts/125/complete
        ↓
Routing selects Complete endpoint
        ↓
Authentication establishes user
        ↓
Authorization checks permission
        ↓
Controller calls CompleteGiftService
        ↓
GiftRepository loads Gift
        ↓
Gift.Complete()
        ↓
SaveChanges
```

If Gift doesn't exist:

EXAMPLE / FLOW

```
GiftNotFoundException
        ↓
GiftNotFoundExceptionHandler
        ↓
404 ProblemDetails
```

If an unexpected infrastructure failure occurs:

EXAMPLE / FLOW

```
unexpected exception
        ↓
global fallback
        ↓
500 ProblemDetails
        ↓
technical details remain in server diagnostics
```

The key principle is:

> **Routing decides where the request goes, middleware controls how the request travels there, authentication and authorization determine who may continue, endpoints translate HTTP into application operations, and centralized error handling translates failures back into safe and consistent HTTP responses.**

QUICK REVISION

Concept	Meaning
Middleware	Request/response pipeline component
`HttpContext`	Current HTTP request context
`HttpRequest`	Incoming request
`HttpResponse`	Outgoing response
`RequestDelegate`	Async request handler delegate
`Use`	Middleware that may call next

Concept	Meaning
`Run`	Terminal middleware
`Map`	Branch pipeline by path
Short-circuit	Stop downstream processing
Middleware order	Request forward, response reverse
Conventional middleware	Usually app-lifetime instance
`IMiddleware`	DI/factory-activated middleware
Routing	Match request to endpoint
Endpoint	Handler + metadata
Route template	URL matching pattern
Route parameter	Dynamic path value
Route constraint	Route disambiguation rule
Route precedence	Specific routes win
`MapControllers`	Attribute-routed controllers
`MapGroup`	Group Minimal API routes
`UseExceptionHandler`	Global exception middleware
`IExceptionHandler`	Central exception handler abstraction
`ProblemDetails`	Standard API error payload
`UseStatusCodePages`	Body for empty error statuses
`Response.HasStarted`	Whether response has begun

FINAL END-TO-END REQUEST FLOW

EXAMPLE / FLOW

```
CLIENT
   ↓
HTTP REQUEST
   ↓
KESTREL
   ↓
HttpContext created
   ↓
GLOBAL EXCEPTION HANDLER
   ↓
HTTPS / FORWARDED / OTHER EDGE MIDDLEWARE
   ↓
ROUTING
   ↓
Route templates matched
   ↓
Constraints applied
   ↓
Endpoint selected
   ↓
ENDPOINT-AWARE MIDDLEWARE
   ↓
```

```
CORS
  ↓
AUTHENTICATION
  ↓
AUTHORIZATION
  ↓
CUSTOM REQUEST MIDDLEWARE
  ↓
SELECTED ENDPOINT
  ↓
Controller / Minimal API
  ↓
Application Service
  ↓
Domain Model
  ↓
Repository / EF Core
  ↓
SQL Server
```

Success:

EXAMPLE / FLOW

```
Application result
    ↓
HTTP result
    ↓
Response travels outward
    ↓
Middleware after-next logic
    ↓
Kestrel
    ↓
Client
```

Known failure:

EXAMPLE / FLOW

```
GiftNotFoundException
    ↑
stack unwinds
    ↑
IExceptionHandler
    ↓
404 ProblemDetails
    ↓
Client
```

Unexpected failure:

EXAMPLE / FLOW

```
Unexpected Exception
    ↑
Exception Handler
    ↓
500 ProblemDetails
    ↓
Trace identifier
    ↓
Detailed diagnostics remain server-side
```

Final mental model:

EXAMPLE / FLOW

```
MIDDLEWARE
    =
How requests move through the app

ROUTING
    =
Which endpoint should handle the request

ENDPOINT
    =
What request handler executes

ERROR HANDLING
    =
How failures become controlled HTTP responses
```

The central rule to remember is:

> **ASP.NET Core is an ordered asynchronous pipeline. Every middleware placement has consequences. Route selection, authentication, authorization, error handling, and endpoint execution are separate responsibilities, and a production-quality API deliberately composes them in the correct order rather than putting all request-processing logic inside controllers.**

Next roadmap topic: Topic 39 — ASP.NET Core Web APIs and REST.